

Bagian A.1 Lifecycle welcoming socket vs connection socket

src/TCPServer.py: pada port 12000,

```
```python
serverSocket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
serverSocket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
serverSocket.bind((args.host, args.port))
serverSocket.listen(args.backlog)
while True:
 connectionSocket, addr = serverSocket.accept()
 t = threading.Thread(target=handle_client,
 args=(connectionSocket, addr), daemon=True)
 t.start()
```
```

Welcoming socket (*serverSocket*) dibuat sekali saat server start, di-*bind* ke port 12000 dan *listen*, lalu hidup selama server berjalan. Tugasnya hanya menerima permintaan koneksi (SYN) dan menyelesaikan 3-way handshake; ia tidak pernah dipakai untuk mengirim data aplikasi. Ditutup hanya ketika server dimatikan.

Connection socket (*connectionSocket*) dibuat oleh *accept()* untuk setiap client yang handshake-nya selesai. Soket ini khusus untuk satu client, diidentifikasi oleh 4-tuple (IP sumber, port sumber, IP tujuan, port tujuan), dan dipakai untuk seluruh pertukaran data. Soket ditutup (*close()*) saat client keluar atau koneksi putus, sementara welcoming socket tetap menerima client baru. Karena setiap connection socket diurus thread sendiri, *recv()* yang blocking pada satu client tidak menghambat client lain.

Bagian A.2 UDP Pinger

src/UDPServer.py: pada port 12001

```
"""
serverSocket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
serverSocket.bind(("", args.port))
serverSocket.settimeout(1.0) # agar Ctrl+C responsif
log(f"UDP server siap di port {args.port} | fd={serverSocket.fileno()} "
    f"| loss={args.loss} delay<={args.delay}s")

seen_clients = set()
try:
    while True:
        try:
            message, clientAddress = serverSocket.recvfrom(2048) # 1
            datagram utuh
        except socket.timeout:
            continue
        except ConnectionResetError: # Windows: ICMP unreachable dari
            balasan lama
"""
```

```

        continue

    if clientAddress not in seen_clients:
        seen_clients.add(clientAddress)
        log(f"Client baru {clientAddress} | total alamat
unik={len(seen_clients)} "
        f"(tetap 1 soket)")

    text = message.decode(errors="replace")
    if random.random() < args.loss:
        log(f"DROP (simulasi) dari {clientAddress}: {text}")
        continue
    if args.delay > 0:
        time.sleep(random.uniform(0, args.delay))

    reply = text.replace("PING", "PONG", 1)
    serverSocket.sendto(reply.encode(), clientAddress)
    log(f"{clientAddress} -> {text} | balas: {reply}")
"""

```

Output statistik RTT

```

PS D:\Bingo\MCs\Jaringan Komputer Lanjut\Github\Projek-TCP-UDP\src> py UDPClient.py --host 192.168.10.205
Port lokal client: 51831 (ephemeral dari OS)

Ping 1: Request timed out
Ping 2: Request timed out
Ping 3: Request timed out
Ping 4: balasan='PONG 4 1789917478.575490' RTT=247.612 ms | EstimatedRTT=247.612 ms
Ping 5: balasan='PONG 5 1789917479.324814' RTT=198.877 ms | EstimatedRTT=240.520 ms
Ping 6: balasan='PONG 6 1789917480.018588' RTT=34.663 ms | EstimatedRTT=214.788 ms
Ping 7: balasan='PONG 7 1789917480.554955' RTT=189.972 ms | EstimatedRTT=211.686 ms
Ping 8: balasan='PONG 8 1789917481.248854' RTT=21.030 ms | EstimatedRTT=187.854 ms
Ping 9: balasan='PONG 9 1789917481.771096' RTT=303.005 ms | EstimatedRTT=202.248 ms
Ping 10: balasan='PONG 10 1789917482.575623' RTT=214.377 ms | EstimatedRTT=203.764 ms

--- Statistik UDP Ping ---
Ter kirim=10, Diterima=7, Hilang=3 (30.0% packet loss)
RTT min/avg/max = 21.030 / 171.648 / 303.005 ms
EstimatedRTT = 203.764 ms, DevRTT = 95.736 ms, TimeoutInterval = 586.707 ms
PS D:\Bingo\MCs\Jaringan Komputer Lanjut\Github\Projek-TCP-UDP\src>

```

Bagian B.1 Byte-Stream vs Message Boundary

TCP (*SOCK_STREAM*) menyediakan layanan aliran byte yang andal dan berurutan. *send()* hanya menyalin byte ke *send buffer* kernel; TCP bebas memotong atau menggabungkan byte tersebut menjadi segmen sesuai MSS, window, dan algoritma Nagle. Di penerima, byte masuk ke *receive buffer* dan *recv()* mengembalikan berapa pun byte yang tersedia (hingga ukuran buffer yang diminta). Tidak ada informasi di header TCP yang menandai "akhir pesan aplikasi", sehingga batas antar panggilan *send()* hilang.

UDP (*SOCK_DGRAM*) memperlakukan setiap *sendto()* sebagai satu datagram independen dengan header dan panjang sendiri. *recvfrom()* selalu mengembalikan tepat satu datagram utuh (atau

terpotong/dibuang jika buffer terlalu kecil), sehingga batas pesan terjaga — meski datagram bisa hilang, terduplikasi, atau tiba tidak berurutan.

Dampak 3 kali send() tanpa delay: penerima bisa mendapat ketiganya dalam satu recv() (*coalescing*), atau satu pesan terbelah ke dua recv() (*partial read*). Percobaan *tcp_raw_demo.py* membuktikan: tiga pesan diterima sebagai satu byte string *HaloApa kabar?Selesai*, sehingga aplikasi tidak bisa tahu di mana pesan berakhir.

Protokol aplikasi memakai header fixed-length 5 byte (TYPE 1 byte + LENGTH 4 byte, network byte order). Penerima membaca tepat 5 byte, lalu tepat LENGTH byte menggunakan *recv_exact()* yang mengulang *recv()* hingga jumlah byte terpenuhi. Hasilnya, dengan --demo-burst server mencatat tiga pesan terpisah. Panjang-prefiks dipilih dibanding delimiter karena aman untuk data biner (file) yang mungkin mengandung karakter delimiter.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS D:\Bingo\MCs\Jaringan Komputer Lanjut\Github\Projek-TCP-UDP\src> py TCPServer.py
[21:23:46] [MainThread] Welcoming socket LISTEN di port 12000 | fd=324
[21:27:54] [client-42988] TERHUBUNG ('192.168.10.10', 42988) | fd connectionSocket=328 | client aktif=1 | thread aktif=2
[21:28:30] [client-42988] CMD dari 192.168.10.10:42988: /nick Fachry
[21:28:49] [client-42988] CMD dari Fachry: /users
[21:28:59] [client-42988] TEXT dari Fachry: 'Pesan-1: Halo'
[21:28:59] [client-42988] TEXT dari Fachry: 'Pesan-2: Apa kabar?'
[21:28:59] [client-42988] TEXT dari Fachry: 'Pesan-3: Selesai'
[21:29:13] [client-42988] CMD dari Fachry: /list
[21:29:22] [client-42988] CMD dari Fachry: /send
```

| No. | Time | Source | Destination | Protocol | Length | Info |
|------|---------------|---------------|---------------|----------|--------|--|
| 2679 | 147.178440900 | 10.177.162.43 | 10.177.162.38 | TCP | 54 | 12000 → 57216 [ACK] Seq=68 Ack=11 Win=65536 Len=0 |
| 2680 | 147.178301400 | 10.177.162.43 | 10.177.162.38 | TCP | 90 | 12000 → 57216 [PSH, ACK] Seq=68 Ack=11 Win=65536 Len=36 |
| 2684 | 147.238965300 | 10.177.162.38 | 10.177.162.43 | TCP | 54 | 57216 → 12000 [ACK] Seq=11 Ack=104 Win=65536 Len=0 |
| 2685 | 152.432060700 | 10.177.162.38 | 10.177.162.43 | TCP | 70 | 57216 → 12000 [PSH, ACK] Seq=11 Ack=104 Win=65536 Len=16 |
| 2686 | 152.483661400 | 10.177.162.43 | 10.177.162.38 | TCP | 87 | 12000 → 57216 [PSH, ACK] Seq=104 Ack=27 Win=65536 Len=33 |
| 2696 | 152.937830500 | 10.177.162.38 | 10.177.162.43 | TCP | 54 | 57216 → 12000 [ACK] Seq=27 Ack=137 Win=65536 Len=0 |
| 3094 | 157.861880600 | 10.177.162.38 | 10.177.162.43 | TCP | 72 | 57216 → 12000 [PSH, ACK] Seq=27 Ack=137 Win=65536 Len=18 |
| 3095 | 157.861766400 | 10.177.162.43 | 10.177.162.38 | TCP | 77 | 12000 → 57216 [PSH, ACK] Seq=137 Ack=49 Win=65536 Len=23 |
| 3096 | 157.861856300 | 10.177.162.38 | 10.177.162.43 | TCP | 80 | 57216 → 12000 [PSH, ACK] Seq=45 Ack=108 Win=65536 Len=43 |
| 3697 | 157.864149500 | 10.177.162.43 | 10.177.162.38 | TCP | 81 | 12000 → 57216 [PSH, ACK] Seq=108 Ack=96 Win=65536 Len=28 |
| 3698 | 157.142150300 | 10.177.162.38 | 10.177.162.43 | TCP | 54 | 57216 → 12000 [ACK] Seq=90 Ack=189 Win=65536 Len=0 |
| 3699 | 157.156840300 | 10.177.162.43 | 10.177.162.38 | TCP | 80 | 12000 → 57216 [PSH, ACK] Seq=180 Ack=99 Win=65536 Len=26 |
| 3100 | 157.245646200 | 10.177.162.38 | 10.177.162.43 | TCP | 54 | 57216 → 12000 [ACK] Seq=90 Ack=215 Win=65536 Len=0 |

Bagian B.2 Skalabilitas Socket

TCP dengan N client: proses server membuka **N + 1** soket — 1 welcoming socket + N connection socket. Secara total file descriptor proses juga mencakup stdin/stdout/stderr (0,1,2) dan file lain yang terbuka, jadi sekitar N + 4. Semua connection socket memakai port lokal yang sama (12000); OS membedakannya lewat 4-tuple, sehingga IP/port client yang berbeda menghasilkan soket berbeda. Setiap soket membawa TCB, send buffer, dan receive buffer sendiri, serta pada desain ini 1 thread

(dengan stack sendiri). Batas praktisnya adalah *ulimit -n* (fd per proses), memori kernel, dan jumlah thread .

UDP hanya 1 soket: UDP bersifat *connectionless*; demultiplexing di OS hanya berdasarkan IP dan port tujuan (2-tuple), sehingga seluruh datagram ke port 12001 masuk ke satu soket yang sama. Alamat pengirim diketahui dari nilai kembali *recvfrom()*, dan balasan dikirim dengan *sendto(..., clientAddress)*. Log UDP Server menunjukkan banyak alamat client unik, tetapi tetap satu fd.

Implikasi: memori OS jauh lebih hemat (satu receive buffer, tanpa state koneksi per client), dan hanya satu port yang perlu di-bind. Konsekuensinya, jika trafik tinggi buffer tunggal itu bisa penuh sehingga datagram dibuang, dan state per client (sesi, urutan, keandalan) harus dikelola sendiri oleh aplikasi.

Bagian B.3 Evolusi Transport dan QUIC

Mengapa di atas UDP?:

1. *Deployability / ossification*: NAT, firewall, dan middlebox di Internet umumnya hanya meloloskan TCP dan UDP. Protokol transport baru dengan nomor protokol IP baru (seperti SCTP) sering diblok, sehingga praktis tidak bisa dipakai luas.

2. *Implementasi di user space*: TCP diimplementasikan di kernel OS, sehingga perubahan butuh pembaruan OS di miliaran perangkat. UDP hanya memberi layanan minimal (multiplexing port + checksum), sehingga QUIC dapat mengimplementasikan keandalan, congestion control, dan kriptografi sendiri di aplikasi/library dan diperbarui secepat merilis browser.

3. *Keterbatasan TCP*: menumpang di TCP berarti mewarisi HOL blocking level byte-stream, handshake TCP yang terpisah dari TLS, serta header yang terbaca dan diubah middlebox. QUIC menggabungkan handshake transport dan TLS 1.3 (1-RTT, bahkan 0-RTT untuk koneksi ulang), mengenkripsi hampir seluruh header, dan mendukung *connection migration* lewat Connection ID.

Penyelesaian HOL blocking: Pada HTTP/2 di atas TCP, banyak stream (objek web) dimultipleks di satu byte-stream. Jika satu segmen TCP hilang, semua byte setelahnya tertahan di receive buffer sampai retransmisi tiba, sehingga semua stream tertunda walaupun datanya milik stream lain. QUIC memultipleks beberapa *stream* dalam satu koneksi, dan keandalan serta pengurutan dijaga **per stream** (setiap STREAM frame memiliki stream ID dan offset sendiri). Paket QUIC yang hilang hanya menahan stream yang datanya ada di paket itu; stream lain tetap diserahkan ke aplikasi. Nomor paket juga selalu naik (tidak dipakai ulang saat retransmisi), sehingga pengukuran RTT dan deteksi loss lebih akurat.

Bagian B.4 Bind Eksplisit pada Client

clientSocket.bind('', 5432) membuat OS tidak memilih ephemeral port; semua datagram dikirim dengan port sumber 5432 (terlihat di Wireshark dan di log server (... , 5432)). Port ≥ 1024 tidak memerlukan hak administrator.

Apakah server masih bisa membalas? Ya. Server membaca alamat pengirim dari *recvfrom()* dan membalas ke (*IP_client*, 5432), sehingga balasan sampai ke soket client tersebut. Server tidak peduli

apakah port itu ephemeral atau tetap. (Jika melewati NAT, port bisa ditranslasi, tetapi NAT tetap memetakan balasan dengan benar.)

Konflik dua instans: Instans kedua gagal saat *bind()* dengan *WinError 10048*, karena pasangan (IP, port 5432) sudah dipakai instansi pertama. Jika programmer memaksa berbagi port dengan *SO_REUSEADDR/SO_REUSEPORT*, dua soket bisa ter-bind tetapi kernel yang menentukan soket mana yang menerima tiap datagram balasan, sehingga balasan bisa salah alamat, dianggap loss oleh instansi lain, dan menghasilkan RTT yang salah. Pemeriksaan nomor urut di *UDPClient.py* mengurangi salah hitung, tetapi tidak mencegah paket "dicuri". Masalah lain: port tetap lebih mudah bentrok dengan layanan lain dan lebih mudah ditebak. Karena itu client sebaiknya memakai ephemeral port.

```
Ping 7: balasan='PONG 7 1789917608.326505' RTT=217.511 ms | EstimatedRTT=164.563 ms
Ping 8: balasan='PONG 8 1789917609.045928' RTT=22.511 ms | EstimatedRTT=146.807 ms
Ping 9: balasan='PONG 9 1789917609.570950' RTT=13.128 ms | EstimatedRTT=130.097 ms
Ping 10: balasan='PONG 10 1789917610.086339' RTT=195.908 ms | EstimatedRTT=138.323 ms

--- Statistik UDP Ping ---
Terkirim=10, Diterima=10, Hilang=0 (0.0% packet loss)
RTT min/avg/max = 13.128 / 159.562 / 317.117 ms
EstimatedRTT = 138.323 ms, DevRTT = 98.253 ms, TimeoutInterval = 531.334 ms
PS D:\Bingo\MCs\Jaringan Komputer Lanjut\Github\Projek-TCP-UDP\src> py UDPClient.py --host 192.168.10.205 --bind-port 5432
[!] Gagal bind ke port 5432: [WinError 10048] Only one usage of each socket address (protocol/network address/port) is normally permitted
Kemungkinan port sudah dipakai instansi client lain.
PS D:\Bingo\MCs\Jaringan Komputer Lanjut\Github\Projek-TCP-UDP\src> |
```